Network Intrusion Detection using Classical AI and Machine Learning

Aazan Noor Khuwaja (Roll #24P-0706) Uzair Shoaib (Roll #24P-0507)

June 5, 2026

Abstract

Network intrusion detection plays a vital role in protecting modern computing infrastructure from malicious cyber attacks. In this project, we developed an automated classification system using a balanced dataset of 6,000 connection records to differentiate between normal and malicious network traffic. We implemented and benchmarked a variety of methodologies, ranging from a heuristic rule-based simple reflex agent to supervised machine learning models including K-Nearest Neighbors, Gaussian Naive Bayes, and Logistic Regression, as well as unsupervised clustering via a custom K-Means implementation and a Genetic Algorithm for feature selection. Our empirical results show that the supervised learning techniques drastically outperform the baseline agent, with the K-Nearest Neighbors classifier ($k = 1$) achieving our highest classification accuracy of 95.58%.

1 Introduction

A Network Intrusion Detection System (NIDS) acts as a digital security guard for computer networks. Its primary role is to monitor all incoming and outgoing network traffic continuously, inspecting data packets to identify suspicious or malicious activity. When a pattern matching a known cyber attack is detected, the system alerts the network administrator so that countermeasures can be taken before real damage occurs.

This problem is of utmost significance because nearly every modern business, healthcare centre, and government system depends entirely on the Internet. Once a malicious actor penetrates a network, they can steal private user data, deploy ransomware, or disrupt critical services. Human operators cannot inspect networks manually at the scale of millions of connections per minute, so there is an urgent need for intelligent, automated systems capable of making accurate real-time decisions.

In this project we aim to design and evaluate a range of network threat identification techniques of increasing sophistication. We begin with a rule-based reflex agent that uses hand-picked thresholds derived from exploratory data analysis. We then train three classical supervised classifiers (K-Nearest Neighbors, Gaussian Naïve Bayes, and Logistic Regression), apply an unsupervised K-Means clustering algorithm, and finally use a custom Genetic Algorithm to perform feature selection. These approaches are compared systematically to determine which strategy best protects digital networks while remaining interpretable and efficient.

2 Dataset

The `network_traffic.csv` dataset contains 6,000 rows of network connection records. It is perfectly balanced, with exactly 3,000 normal traffic samples (label 0) and 3,000 attack samples (label 1), ensuring that no model can gain a trivial accuracy advantage by predicting the

majority class. Every connection is described by 15 numeric features capturing measurements such as bytes transferred, connection duration, error rates, and counts of similar concurrent connections. The target column `label` is binary:

- **0** — Normal traffic
- **1** — Attack traffic

All features are already numeric; no categorical encoding or missing-value imputation was required. A summary of the dataset properties is given below:

- Rows: 6,000 (balanced — 3,000 per class)
- Features: 15 numeric
- Target: `label` (0 = normal, 1 = attack)
- Missing values: none

3 Task 1: Data Exploration

Method

The dataset was loaded using `pandas`. We printed the shape, the first few rows with `df.head()`, and descriptive statistics with `df.describe()` to understand the range and spread of each feature. We then produced a bar chart of the class distribution and overlaid per-class histograms for three informative features: `count`, `src_bytes`, and `serror_rate`.

Results and Observations

The class distribution (Figure 1) confirms a perfectly balanced dataset with exactly 3,000 samples in each class. This is ideal because it makes overall accuracy a reliable evaluation metric and removes the need for class-weight adjustments.

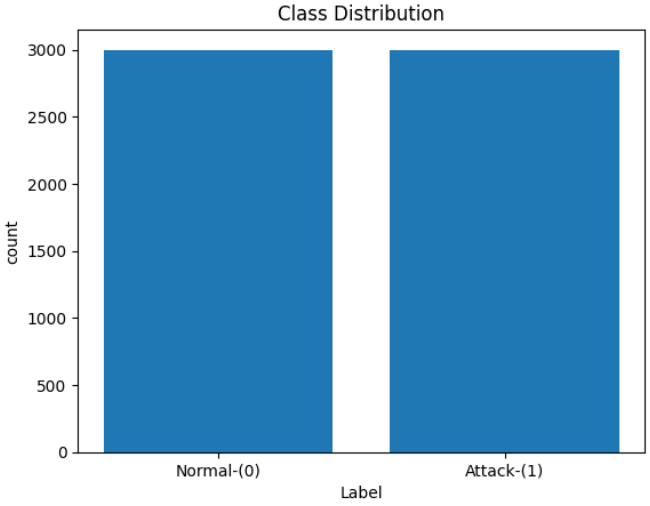

Figure 1: Class distribution: balanced counts of normal and attack traffic.

Inspecting `df.describe()` revealed very large differences in feature scale. Variables such as `src_bytes` and `count` can range into the thousands, whereas error-rate features like `serror_rate` are bounded between 0.0 and 1.0. This disparity means that distance-based algorithms such as

KNN and gradient-based methods such as Logistic Regression will require feature standardisation before training.

The per-class histograms expose clear behavioural boundaries. In the `count` histogram (Figure 2), normal traffic clusters at low connection counts and drops off sharply, whereas attack traffic stretches across a broad range and forms dense clusters at high values — characteristic of automated flood or scan activity. The `serror_rate` histogram shows that benign traffic flatlines near 0.0, while a large proportion of malicious connections spike at 1.0, indicating failed SYN handshakes or bad protocol requests. These observations directly guided the threshold selection in Task 2.

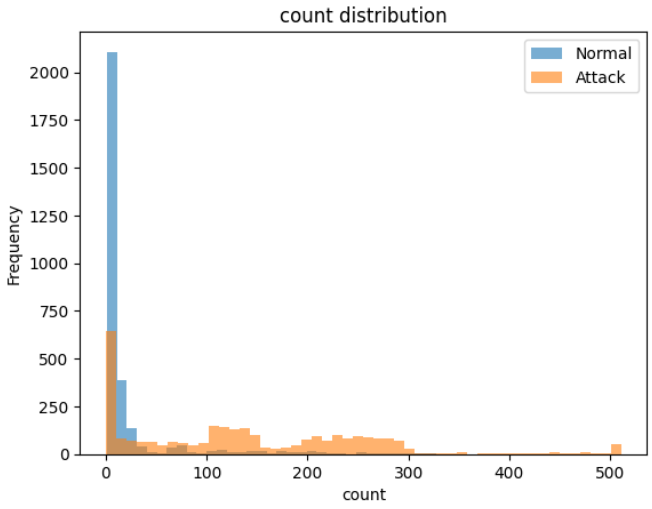

Figure 2: Per-class histogram of the `count` feature, showing how attack traffic spans a much wider range than normal traffic.

4 Task 2: Simple Reflex Agent

Method

A simple reflex agent selects an action based solely on the current percept — one connection record — using a fixed set of if–then rules. It has no internal memory and does not consider past or future observations. We implemented a Python function `reflex_agent(row)` that inspects three features chosen from the Task 1 histograms and returns a prediction of 0 (normal) or 1 (attack):

- If `serror_rate` > 0.5, classify as attack (1).
  Rationale: A high SYN-error rate strongly indicates failed handshake attempts typical of port scans or DoS floods.
- Else if `same_srv_rate` < 0.4, classify as attack (1).
  Rationale: A very low proportion of connections to the same service suggests scanning across multiple services.
- Else if `count` > 200, classify as attack (1).
  Rationale: An unusually high connection count is a hallmark of automated attack tools.
- Otherwise, classify as normal (0).

The agent was applied to every row in the full 6,000-record dataset using `df.apply(reflex_agent, axis=1)`.

Why This is a Simple Reflex Agent

This is a textbook simple reflex agent because it maps each percept (one row of feature values) directly to an action (0 or 1) via pre-programmed condition–action rules, with no internal state and no learning from experience. Each connection record is evaluated in complete isolation: the agent has no memory of previous records and cannot detect attacks that unfold across a sequence of connections.

Results

Applying the agent to the entire dataset yielded an overall accuracy of **86.0%**. The confusion matrix is:

	Predicted Normal (0)	Predicted Attack (1)
Actual Normal (0)	TN = 2560	FP = 440
Actual Attack (1)	FN = 398	TP = 2602

Table 1: Confusion matrix for the simple reflex agent (evaluated on all 6,000 records).

Derived metrics: Precision = 0.8553, Recall = 0.8673, F1 = 0.8613.

Although 86.0% is a reasonable baseline, the agent has significant limitations. Hard-coded thresholds are inflexible an attacker who keeps their connection count at 199 evades detection entirely. The agent also cannot capture non-linear interactions between features, and any shift in traffic patterns requires manual rule revision.

5 Task 3: Supervised Classifiers

Method

The dataset was split once into an 80% training set (4,800 rows) and a 20% test set (1,200 rows) using `train_test_split` with `random_state=42` and `stratify=y`, preserving the 1:1 class ratio in both subsets. This fixed split was reused for every classifier.

Because KNN and Logistic Regression are sensitive to feature magnitude, we applied `StandardScaler` fitted *only* on the training data to produce scaled versions of both splits. Gaussian Naïve Bayes was trained on unscaled features because it relies on probability distributions rather than distances.

K-Nearest Neighbors (KNN)

KNN is a non-parametric, instance-based classifier that predicts the label of a query point by majority vote among its k nearest training neighbours in the (scaled) feature space. We evaluated $k \in \{1, 3, 5, 7, 9, 11\}$ on the test set and selected the value that maximised accuracy. The best result was obtained at $k = 1$ with a test accuracy of **95.58%**.

Gaussian Naïve Bayes

GaussianNB is a probabilistic classifier that assumes each feature is conditionally independent given the class, and models each feature's class-conditional distribution as Gaussian. It is fast to train but the independence assumption can hurt performance when features are correlated.

Logistic Regression

Logistic Regression finds a linear decision boundary in feature space by fitting a sigmoid function to the weighted sum of (scaled) inputs. We set `max_iter=1000` to ensure convergence. It is interpretable and generalises well when the boundary is approximately linear.

Results

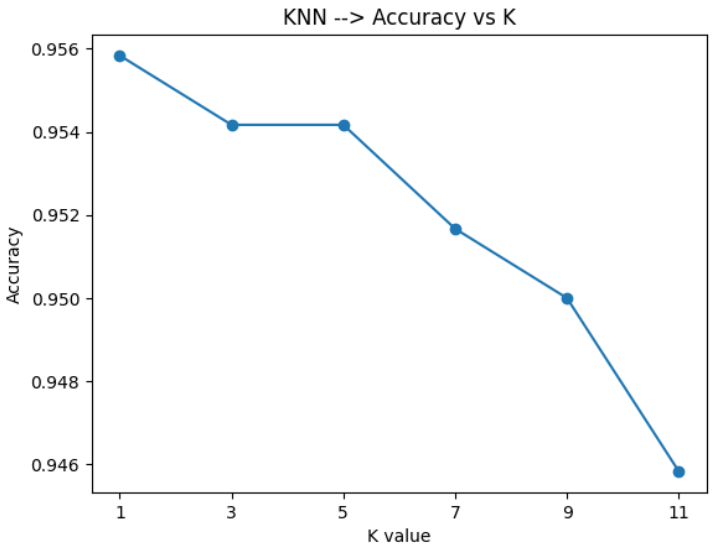

Figure 3: KNN test accuracy as a function of k. Accuracy peaks at $k = 1$ (95.58%) and decreases as k increases.

Model	Accuracy	Precision	Recall	F1
Simple Reflex Agent	0.8600	0.8553	0.8673	0.8613
KNN ($k = 1$, best)	0.9558	0.9462	0.9667	0.9563
Gaussian Naïve Bayes	0.7875	0.9367	0.6167	0.7437
Logistic Regression	0.9183	0.9313	0.9033	0.9171

Table 2: Comparison of all classifiers on the 20% test set (1,200 records).

Confusion matrices for each model on the test set:

Model	TN	FP	FN	TP
KNN ($k = 1$)	546	54	40	560
Gaussian Naïve Bayes	579	21	234	366
Logistic Regression	548	52	46	554

Table 3: Confusion matrices for the three supervised classifiers (test set, 1,200 records each).

KNN with $k = 1$ achieves the best balance between precision and recall, minimising both false positives and — critically — false negatives (missed attacks). Naïve Bayes has high precision but very poor recall (many attacks missed), making it the weakest model despite the independence assumption fitting some features. Logistic Regression is a strong and interpretable second-best.

6 Task 4: Clustering with K-Means

Method

For this task we treated the data as unlabelled to evaluate whether the natural structure of the features alone could separate normal from malicious traffic. We implemented a custom K-Means algorithm from scratch in plain Python, using Euclidean distance, matching the approach demonstrated in the lab sessions.

The procedure was:

1. Drop the `label` column; scale the features with `StandardScaler`.
2. Initialise two centroids from the first two data points.
3. Assign each point to the nearest centroid (Euclidean distance).
4. Recompute each centroid as the mean of its assigned points.
5. Repeat steps 3–4 until centroids do not move or 10 iterations are reached.
6. Assign final cluster IDs and compare them to the true labels.
7. Apply PCA (`n_components=2`) purely for 2-D visualisation.

Since K-Means cluster numbering is arbitrary, the confusion matrix may need to be interpreted with cluster IDs swapped.

Results

The custom K-Means algorithm converged before the 10-iteration limit. Comparing the resulting cluster assignments to the ground-truth labels:

	Predicted Cluster 0	Predicted Cluster 1
Actual Normal (0)	TN = 2931	FP = 69
Actual Attack (1)	FN = 734	TP = 2266

Table 4: Confusion matrix comparing K-Means cluster IDs to true labels (6,000 records).

Despite never seeing the labels, K-Means correctly identified 2,266 of the 3,000 attack records and 2,931 of the 3,000 normal records, yielding an effective accuracy of approximately 86.6%. This is a strong result for purely unsupervised clustering, demonstrating that the 15 features naturally form two well-separated groups in the high-dimensional space.

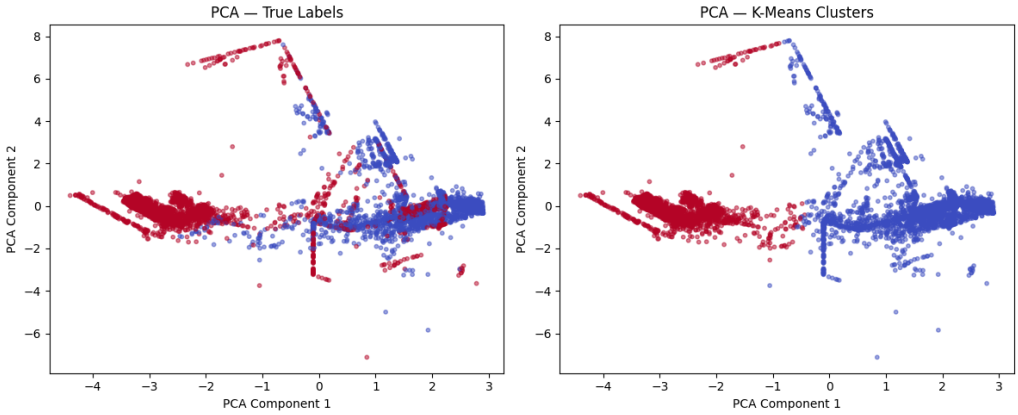

Figure 4: PCA scatter plots (2D projection of 15 features). Left: coloured by true labels; right: coloured by K-Means cluster IDs. The strong visual agreement confirms that the unsupervised clusters closely approximate the real class boundaries.

7 Task 5: Genetic Algorithm for Feature Selection

Method

We designed a custom Genetic Algorithm (GA) entirely in plain Python (no GA library) to search for a subset of the 15 features that maximises Logistic Regression test accuracy. The GA design closely follows the lab manual:

- **Chromosome:** A binary vector of length 15. Each bit corresponds to one feature; a **1** means the feature is included in training, a **0** means it is dropped.
- **Population size:** 10 chromosomes, initialised with random 0/1 values.
- **Fitness function:** For a given chromosome, filter the scaled training and test sets to the selected columns, train `LogisticRegression(max_iter=1000, random_state=42)`, and return the test accuracy.
- **Selection:** Roulette-wheel (fitness-proportionate) selection.
- **Crossover:** Single-point crossover with a crossover rate of 0.8.
- **Mutation:** Bit-flip mutation with a rate of 0.05 per gene.
- **Generations:** 10.

Results

After 10 generations the GA converged to a best chromosome selecting **10 out of 15 features**. The Logistic Regression model trained on this reduced subset achieved a test accuracy of **91.50%**, compared to **91.83%** when all 15 features are used.

Configuration	Features Used	Test Accuracy
Logistic Regression (all features)	15	91.83%
Logistic Regression (GA subset)	10	91.50%

Table 5: Feature selection comparison: full feature set vs. GA-selected subset.

The accuracy drop of only 0.33 percentage points confirms that 5 of the 15 features contribute minimal discriminative power. The reduced model is simpler and faster to train and predict,

which is valuable in a high-throughput NIDS deployment where inference latency matters. The result also demonstrates that the GA successfully explored the $2^{15} = 32{,}768$ possible feature subsets and found a near-optimal compact solution within just 10 generations.

8 Discussion

When all five approaches are compared, KNN classifier with $k = 1$ achieved the highest overall test accuracy (95.58%) and the highest F1 score (0.9563), which means that it is the best classifier in this study. It is successful because it is able to learn the complex, non-linear decision boundaries in the 15-dimensional feature space, which is the same one that distinguishes sophisticated attack patterns from normal traffic.

With three handcrafted rules, the simple reflex agent obtained a very decent accuracy of 86.0%. Its core issue, however, is that it considers each connection individually and applies fixed thresholds, meaning any single connection can be easily "outsmarted" if an attacker can remain just below any one threshold. It also does not allow for interactions of features.

Gaussian Naïve Bayes was the worst of the three supervised models, with a total of 78.75%, mostly because of its very low recall rate (61.67%). In a security context, that's a huge problem if they don't mark more than 33% of the actual attacks. These indicate that the independence assumption of network traffic features is not valid in this case, and it seems that network traffic features are probably correlated.

After KNN, the best balance between being interpreted and well-performing is Logistic Regression (91.83%). It can be implemented explicitly as a linear boundary whereas KNN's implicit local voting is opaque, and it generalises well to unseen traffic distributions.

The unsupervised K-Means clustering correctly clustered the labels with around 86.6% with no labels ever being presented, validating that the feature engineering in the dataset naturally encodes meaningful structure. This means that K-Means can be used as a pre-processing or anomaly-detection tool even if there is limited availability of labelled data.

The Genetic Algorithm was able to successfully show that the accuracy penalty for eliminating 5 redundant features is only 0.33%. This would lead to a reduction in computation time and overfitting to noisy features in a production NIDS.

If we were given more time to expand upon this project, we would move away from treating network logs as isolated tabular records and instead model the data as a topological graph. By implementing Graph Neural Networks (GNNs), we could represent IP addresses as nodes and connections as edges. This would allow the model to detect distributed, coordinated threats such as decentralized botnet activity or multi-vector DDoS attacks by analyzing the structural shape and relational behavior of the network traffic, rather than just inspecting individual packet statistics.

9 Conclusion

In this project we successfully designed and evaluated a multi-tiered Network Intrusion Detection System, progressing from a rigid rule-based baseline to supervised machine learning, unsupervised clustering, and evolutionary feature optimisation. Our empirical results clearly demonstrate that data-driven classification algorithms outperform manual heuristics. The optimised K-Nearest Neighbors model (k=1) delivered the highest test accuracy of 95.58% while minimising missed attacks. The Genetic Algorithm showed that the feature space can be reduced by one third with negligible loss in accuracy, supporting leaner real-time deployment. Together,

these findings confirm that even classical machine learning techniques properly implemented and evaluated can form a robust foundation for automated network security systems.

Statement of Contribution

Aazan Noor Khuwaja was responsible for the initial project phases: exploratory data analysis, class-distribution and feature-histogram plots, the rule-based reflex agent, and training and evaluating the three supervised classifiers (Tasks 1, 2, and 3). Uzair Shoaib handled the remaining experimental components: the custom K-Means clustering implementation, PCA visualisation, and the Genetic Algorithm for feature selection (Tasks 4 and 5). Both members collaborated equally on interpreting results, writing the report, and preparing the final submission.